```
function training(ds)

% Split the dataset into sub-datasets based on  category
dsGood = subset(ds, ds.Labels == "good");
dsColor = subset(ds, ds.Labels == "colorMismatch");
dsLength = subset(ds, ds.Labels == "defectiveLength");
dsMalformed = subset(ds, ds.Labels == "malformedMetal");

% Randomly split each sub-dataset into training and validation sets of data
% 80/20 Split
[dsGoodTrain, dsGoodValidation] = splitEachLabel(dsGood, 0.8, "randomized");
[dsColorTrain, dsColorValidation] = splitEachLabel(dsColor, 0.8, "randomized");
[dsLengthTrain, dsLengthValidation] = splitEachLabel(dsLength, 0.8, "randomized");
[dsMalformedTrain, dsMalformedValidation] = splitEachLabel(dsMalformed, 0.8, "randomized");

% Generate the final inclusive datastore to be returned by the function
allTestFiles = [dsGoodTrain.Files; dsColorTrain.Files; dsLengthTrain.Files;
dsMalformedTrain.Files];
allTestLabels = [dsGoodTrain.Labels; dsColorTrain.Labels; dsLengthTrain.Labels;
dsMalformedTrain.Labels];
inclusiveDS = imageDatastore(allTestFiles, 'Labels', allTestLabels);

% Generate the final inclusive validation datastore to be used for
% training the model
allValFiles = [dsGoodValidation.Files; dsColorValidation.Files; dsLengthValidation.Files;
dsMalformedValidation.Files];
allValLabels = [dsGoodValidation.Labels; dsColorValidation.Labels; dsLengthValidation.Labels;
dsMalformedValidation.Labels];
inclusiveValidationDS = imageDatastore(allValFiles, 'Labels', allValLabels);

% determine the number of classes that we have
numClasses = numel(categories(inclusiveDS.Labels));
fprintf("Detected %d classes to learn.\n", numClasses);

% resnet18 requires that every image be 224 x 224 pixels in RGB space.
% This line defines the size of each image for use in this training model.
% It reads 224 pixels long by 224 pixels wide, all three channels of color
% (i.e. "R", "G", "B")
imageSize = [224 224 3];

% Flips/Rotates the training images randomly so that the AI model doesn't
% memorize the exact layout of any one image. It forces the AI model to
% learn the shape of the tubes.
augmenter = imageDataAugmenter(RandXReflection=true, RandYReflection=true);

% Create temporary datastore pipelines for training and validation datasets
% that contain within them the resized images.
augDsTrain = augmentedImageDatastore(imageSize, inclusiveDS, "DataAugmentation",augmenter);
augDsValidation = augmentedImageDatastore(imageSize, inclusiveValidationDS);

% Loads the pre-trained resnet18 and replaces the final layer containing
% within it the preset 1000 classifications.
%
% The following line replaces the default number of classes (i.e. 1000) to
% however many we have.
net = imagePretrainedNetwork("resnet18", NumClasses=numClasses);

% Setup the training parameters for this model.
%
% First parameter represents the mathematical path selected for learning
% how to correct mistakes (there are multiple to choose from)
```

```
%
% MaxEpochs represents the number of times the dataset is parsed through
%
% ValidationData represents the dataset to be used
%
% Validation Frequency represents how often accuracy is tested
%
% InitialLearnRate represents how often the model learns.
%
% Plots parameter graphs in real time each trial tested
%
% Metrics parameter tracks quantity of interest on live plot
options = trainingOptions("adam", ...
    MaxEpochs=25, ...
    MiniBatchSize= 16, ...
    ValidationData = augDsValidation, ...
    ValidationFrequency=10, ...
    InitialLearnRate = 1e-4, ... %intentionally kept low because resnet18 already is highly
trained for image classification
    Plots="training-progress", ...
    Metrics= "accuracy");

% Train the model using crossentropy scoring technique
trainedNet = trainnet(augDsTrain, net, "crossentropy", options);

% Save the trained model and set of variables to your computer for use on
% validation/testing
save("tubeClassifierNet.mat", "trainedNet");

% Save the images used for testing
outputFolder = sprintf("%s/testImages", pwd);
% Deletes the pre-existing folder if one already exists
if exist(outputFolder, "dir")
    rmdir(outputFolder, 's')
end
writeall(inclusiveValidationDS, outputFolder);

end

% CHANGES 7/26/2026 2:50 PM (Ryan)
%
% Added a testImages folder for the testing set. This allows the same set
% to be used between different sessions. Moved the saving to the end to
% prevent accidental overwriting
%
% CHANGES 7/19/2026 11:06 PM (Isaac)
%
% Redeveloped test and validation datasets by creating them based on
% separate sub-datasets based on individual categories.
% This was done to address the issue that there was roughly a 16% chance
% that a given category of images would be excluded from either
% train/validation dataset. This change creates more inclusive test and
% validation datasets and said datasets are now incorporated into the
% training model code.
%
% Added the new "inclusive" test Dataset as a function output for use in
% other parts of the overall program.
```